

Lab – 4

1. Write a Python program to accept five statements related to cryptology from the user and store them in a text file named cryptology.txt. Store each statement on a separate line.

sample statements:

Cryptology is the study of secure communication.

Cryptography focuses on protecting information.

Cryptanalysis focuses on analyzing cryptographic systems.

Plaintext represents the original message.

Ciphertext represents the transformed message.

Cryptanalysis is fun :)

I love Cryptanalysis ;)

```
filename = "cryptology.txt"

with open(filename, "w") as file:
    print("Enter 5 statements related to cryptology:")
    for i in range(1, 6):
        statement = input(f"Statement {i}: ")
        file.write(statement + "\n")

print(f"\nAll statements have been saved to '{filename}'.")

Enter 5 statements related to cryptology:
Statement 1: Cryptology is the study of secure communication.
Statement 2: Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Statement 3: Plaintext represents the original message. Ciphertext represents the transformed message.
Statement 4: Cryptanalysis is fun :)
Statement 5: I love Cryptanalysis ;)

All statements have been saved to 'cryptology.txt'.
```

2. Write a Python program to read cryptology.txt and display its complete contents.

```
filename = "cryptology.txt"

try:
    with open(filename, "r") as file:
        content = file.read()
        print(content, end="")
except FileNotFoundError:
    print(f"Error: The file '{filename}' was not found. Ensure it exists in the same directory.")

Cryptology is the study of secure communication.
Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message. Ciphertext represents the transformed message.
Cryptanalysis is fun :)
I love Cryptanalysis ;)
```

3. Write a program to read cryptology.txt one line at a time and display each line along with its line number.

```
filename = "cryptology.txt"

try:
    with open(filename, "r") as file:
        for line_number, line in enumerate(file, start=1):
            print(f"Line {line_number}: {line.strip()}")
except FileNotFoundError:
    print(f"Error: The file '{filename}' was not found. Ensure it exists in the same directory.")

Line 1: Cryptology is the study of secure communication.
Line 2: Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Line 3: Plaintext represents the original message. Ciphertext represents the transformed message.
Line 4: Cryptanalysis is fun :)
Line 5: I love Cryptanalysis ;)
```

4. write a program to store the output in a file called cryptology_no.txt

```
input_file = "cryptology.txt"
output_file = "cryptology_no.txt"

try:
    with open(input_file, "r") as infile, open(output_file, "w") as outfile:
        for line_number, line in enumerate(infile, start=1):
            outfile.write(f"Line {line_number}: {line.strip()}\n")

        print(f"Successfully processed '{input_file}' and saved output to '{output_file}'.")
except FileNotFoundError:
    print(f"Error: The file '{input_file}' was not found. Ensure it exists in the same directory.")

Successfully processed 'cryptology.txt' and saved output to 'cryptology_no.txt'.
```

5. Write a program to read cryptology.txt and determine:

Number of lines

Number of words

Number of characters

```
filename = "cryptology.txt"

try:
    line_count = 0
    word_count = 0
    char_count = 0

    with open(filename, "r") as file:
        for line in file:
            line_count += 1
            word_count += len(line.split())
            char_count += len(line)

    print(f"--- Statistics for '{filename}' ---")
    print(f"Number of lines:      {line_count}")
    print(f"Number of words:         {word_count}")
    print(f"Number of characters: {char_count}")

except FileNotFoundError:
    print(f"Error: The file '{filename}' was not found. Ensure it exists in the same directory.")

--- Statistics for 'cryptology.txt' ---
Number of lines:      5
Number of words:      36
Number of characters: 294
```

6. Read cryptology_no.txt and count the number of:

Uppercase letters

Lowercase letters

Digits

Spaces

Special characters

```

▶ filename = "cryptology_no.txt"

try:
    uppercase = 0
    lowercase = 0
    digits = 0
    spaces = 0
    special_chars = 0

    with open(filename, "r") as file:
        content = file.read()
        for char in content:
            if char.isupper():
                uppercase += 1
            elif char.islower():
                lowercase += 1
            elif char.isdigit():
                digits += 1
            elif char.isspace():
                spaces += 1
            else:
                special_chars += 1

    print(f"--- Character Counts in '{filename}' ---")
    print(f"Uppercase letters: {uppercase}")
    print(f"Lowercase letters: {lowercase}")
    print(f"Digits: {digits}")
    print(f"Spaces (incl. \n): {spaces}")
    print(f"Special characters: {special_chars}")

except FileNotFoundError:
    print(f"Error: '{filename}' was not found.")

... --- Character Counts in 'cryptology_no.txt' ---
Uppercase letters: 13
Lowercase letters: 255
Digits: 5
Spaces (incl. \n): 46
Special characters: 14

```

7. Write a program that accepts a word from the user and searches for that word in cryptology.txt.

```

filename = "cryptology.txt"
search_word = input("Enter a word to search for: ").strip()

try:
    found = False
    with open(filename, "r") as file:
        for line_num, line in enumerate(file, start=1):
            # Case-insensitive substring search
            if search_word.lower() in line.lower():
                print(f"Line {line_num}: {line.strip()}")
                found = True

    if not found:
        print(f"The word '{search_word}' was not found in '{filename}'.")

except FileNotFoundError:
    print(f"Error: '{filename}' was not found.")

Enter a word to search for: secure
Line 1: Cryptology is the study of secure communication.

```

8. Read cryptology.txt and display only the lines containing the word message.

```

filename = "cryptology.txt"
target_word = "message"

try:
    print(f"--- Lines containing '{target_word}' ---")
    with open(filename, "r") as file:
        for line in file:
            if target_word in line.lower():
                print(line.strip())

except FileNotFoundError:
    print(f"Error: '{filename}' was not found.")

--- Lines containing 'message' ---
Plaintext represents the original message. Ciphertext represents the transformed message.

```

9. Write a Python program to read cryptology.txt, convert all alphabetic characters to uppercase, and store the result in a new file called normalized.txt.


```

input_file = "cryptology.txt"
output_file = "normalized.txt"

try:
    with open(input_file, "r") as infile, open(output_file, "w") as outfile:
        content = infile.read()
        outfile.write(content.upper())

    print(f"Successfully normalized content to uppercase in '{output_file}'.")

except FileNotFoundError:
    print(f"Error: '{input_file}' was not found.")

Successfully normalized content to uppercase in 'normalized.txt'.

```

10. Write a program to read the file and print only the lines containing only alphabetic characters from the cryptology_no.txt file.

```

filename = "cryptology_no.txt"

try:
    print("--- Strictly Alphabetic Lines ---")
    found = False
    with open(filename, "r") as file:
        for line in file:
            clean_line = line.strip()
            # str.isalpha() returns True ONLY if all characters are letters (no spaces, digits, or punctuation)
            if clean_line.isalpha():
                print(clean_line)
                found = True

    if not found:
        print("No lines contain strictly alphabetic characters (lines in cryptology_no.txt contain numbers and punctuation).")

except FileNotFoundError:
    print(f"Error: '{filename}' was not found.")

--- Strictly Alphabetic Lines ---
No lines contain strictly alphabetic characters (lines in cryptology_no.txt contain numbers and punctuation).

```

11. Write a program to get the string from the user and reverse the string.

```

user_string = input("Enter a string to reverse: ")
reversed_string = user_string[::-1]

print(f"Reversed string: {reversed_string}")

Enter a string to reverse: secure
Reversed string: eruces

```

12. Improve the above code to reverse the string from the file.

```

filename = "cryptology.txt"

try:
    print("--- Reversing Each Line of 'cryptology.txt' ---")
    with open(filename, "r") as file:
        for line in file:
            clean_line = line.strip()
            print(clean_line[::-1])

except FileNotFoundError:
    print(f"Error: '{filename}' was not found.")

--- Reversing Each Line of 'cryptology.txt' ---
.noitacinummoc eruces fo yduts eht si ygotlpyrC
.smetsys cihpargotpyrc gnizylana no sesucof sisylanatpyrC .noitamrofni gnitcetorp no sesucof yhpargotpyrC
.egassem demrofsnart eht stneserper txetrehpiC .egassem lanigiro eht stneserper txetnialP
): nuf si sisylanatpyrC
); sisylanatpyrC evol I

```

13. Simulate the tac command in Python.

```

filename = "cryptology.txt"

try:
    with open(filename, "r") as file:
        lines = file.readlines()

    print(f"--- 'tac' Output for '{filename}' ---")
    for line in reversed(lines):
        print(line.strip())

except FileNotFoundError:
    print(f"Error: '{filename}' was not found.")

--- 'tac' Output for 'cryptology.txt' ---
I love Cryptanalysis ;)
Cryptanalysis is fun :)
Plaintext represents the original message. Ciphertext represents the transformed message.
Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Cryptology is the study of secure communication.

```

or

Create a Python program that accepts five lines of text related to cryptology from the user and stores them in input.txt. Read the file and determine the number of lines, words, alphabetic characters, digits, spaces, and special characters. Convert the text to uppercase and store the processed contents in processed.txt. Finally, read and display the contents of processed.txt.

```

input_filename = "input.txt"
processed_filename = "processed.txt"

# --- 1. Accept 5 lines of text from the user and store in input.txt ---

print("Enter 5 lines related to cryptology:")

with open(input_filename, "w") as infile:

```

```
for i in range(1, 6):  
    line = input(f"Line {i}: ")  
    infile.write(line + "\n")  
  
# --- 2. Read input.txt and calculate character and word statistics ---  
  
line_count = 0  
word_count = 0  
alpha_count = 0  
digit_count = 0  
space_count = 0  
special_count = 0  
  
with open(input_filename, "r") as infile:  
    content = infile.read()  
  
# Split into lines to calculate line and word counts  
lines = content.splitlines()  
line_count = len(lines)  
  
for line in lines:  
    word_count += len(line.split())  
  
# Iterate over raw content to evaluate character types  
for char in content:  
    if char.isalpha():  
        alpha_count += 1  
    elif char.isdigit():  
        digit_count += 1  
    elif char.isspace():  
        space_count += 1
```



```

else:

    special_count += 1

print("\n--- Statistics for 'input.txt' ---")
print(f"Number of lines:          {line_count}")
print(f"Number of words:          {word_count}")
print(f"Number of alphabetic characters: {alpha_count}")
print(f"Number of digits:          {digit_count}")
print(f"Number of spaces (incl. \n):  {space_count}")
print(f"Number of special characters:  {special_count}")

# --- 3. Convert content to uppercase and store in processed.txt ---
with open(processed_filename, "w") as outfile:

    outfile.write(content.upper())

print(f"\nProcessed text successfully written to '{processed_filename}'.")

# --- 4. Read and display the contents of processed.txt ---
print(f"\n--- Contents of '{processed_filename}' ---")
with open(processed_filename, "r") as outfile:

    print(outfile.read(), end="")

```

```

... Enter 5 lines related to cryptology:
Line 1: Cryptology is the study of secure communication.
Line 2: Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Line 3: Plaintext represents the original message. Ciphertext represents the transformed message.
Line 4: Cryptanalysis is fun :)
Line 5: I love Cryptanalysis ;)

--- Statistics for 'input.txt' ---
Number of lines:          5
Number of words:          36
Number of alphabetic characters: 248
Number of digits:         0
Number of spaces (incl. \n): 37
Number of special characters: 9

Processed text successfully written to 'processed.txt'.

--- Contents of 'processed.txt' ---
CRYPTOLOGY IS THE STUDY OF SECURE COMMUNICATION.
CRYPTOGRAPHY FOCUSES ON PROTECTING INFORMATION. CRYPTANALYSIS FOCUSES ON ANALYZING CRYPTOGRAPHIC SYSTEMS.
PLAINTEXT REPRESENTS THE ORIGINAL MESSAGE. CIPHERTEXT REPRESENTS THE TRANSFORMED MESSAGE.
CRYPTANALYSIS IS FUN :)
I LOVE CRYPTANALYSIS ;)

```

Level 2: Regex

security_data.txt:

User alice logged in from 192.168.10.15

User bob logged in from 10.10.5.21

Contact admin@securitylab.com

Failed login attempts: 5

Session ID: SEC-2026-1045

Hash: a94f3c2d8e71b05f

Port 443 connection established

User admin failed authentication

1. Write a Python program to read security_data.txt and use a regular expression to identify and display all IPv4-address-like patterns present in the file.

```
import re

with open("security_data.txt", "r") as file:
    content = file.read()
    ip_addresses = re.findall(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', content)

print("IPv4 Addresses:", ip_addresses)
# Output: ['192.168.10.15', '10.10.5.21']

IPv4 Addresses: ['192.168.10.15', '10.10.5.21']
```

2. Write a Python program to identify and display all email-address-like patterns present in security_data.txt.

```
import re

with open("security_data.txt", "r") as file:
    content = file.read()
    emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', content)

print("Email Addresses:", emails)
# Output: ['admin@securitylab.com']

Email Addresses: ['admin@securitylab.com']
```

3. Write a Python program to extract session IDs that follow the format SEC-YYYY-NNNN, where YYYY and NNNN represent four digits.

```
import re

with open("security_data.txt", "r") as file:
    content = file.read()
    session_ids = re.findall(r'\bSEC-\d{4}-\d{4}\b', content)

print("Session IDs:", session_ids)
# Output: ['SEC-2026-1045']

Session IDs: ['SEC-2026-1045']
```

4. Write a Python program to identify all hexadecimal sequences containing the characters 0–9, a–f, or A–F.

```
import re

with open("security_data.txt", "r") as file:
    content = file.read()
    # Matches hex tokens (like hashes) containing 0-9 and a-f / A-F
    hex_sequences = re.findall(r'\b[0-9a-fA-F]{8,}\b', content)

print("Hexadecimal Sequences:", hex_sequences)
# Output: ['a94f3c2d8e71b05f']

Hexadecimal Sequences: ['a94f3c2d8e71b05f']
```

5. Write a Python program to read security_data.txt and display all lines containing the word failed, irrespective of uppercase or lowercase.

```
import re

print("Lines containing 'failed':")
with open("security_data.txt", "r") as file:
    for line in file:
        if re.search(r'failed', line, re.IGNORECASE):
            print(line.strip())

Lines containing 'failed':
Failed login attempts: 5
User admin failed authentication
```

6. Write a Python program to identify and extract the port number from statements of the form Port 443 connection established.

```
import re

with open("security_data.txt", "r") as file:
    content = file.read()
    ports = re.findall(r'Port\s+(\d+)\s+connection established', content)

print("Extracted Port Numbers:", ports)

Extracted Port Numbers: ['443']
```

or

Write a Python program to read the contents of security_data.txt and use regular expressions to extract IP-address-like patterns, email addresses, session IDs, and hexadecimal sequences. Store the identified patterns in a new file named pattern_results.txt.

```

import re

input_file = "security_data.txt"
output_file = "pattern_results.txt"

# Define Regex Patterns
patterns = {
    "IPv4 Addresses": r'\b(?:\d{1,3}\.){3}\d{1,3}\b',
    "Email Addresses": r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b',
    "Session IDs": r'\bSEC-\d{4}-\d{4}\b',
    "Hexadecimal Sequences": r'\b[0-9a-fA-F]{8,}\b'
}

try:
    with open(input_file, "r") as infile:
        content = infile.read()

    with open(output_file, "w") as outfile:
        outfile.write("=====\n")
        outfile.write("      EXTRACTED PATTERN RESULTS      \n")
        outfile.write("=====\\n\\n")

        for label, regex in patterns.items():
            matches = re.findall(regex, content)
            outfile.write(f"[{label}]\n")
            if matches:
                for match in matches:
                    outfile.write(f"  - {match}\n")
            else:
                outfile.write("  - None found\n")
            outfile.write("\n")

        print(f"Successfully extracted patterns and saved results to '{output_file}'.")

except FileNotFoundError:
    print(f"Error: '{input_file}' was not found. Please create the file first.")

... Successfully extracted patterns and saved results to 'pattern_results.txt'.

```



```
filename = "pattern_results.txt"

try:
    with open(filename, "r") as file:
        content = file.read()
        print(content, end="")
except FileNotFoundError:
    print(f"Error: The file '{filename}' was not found. Ensure it exists in the same directory.")
```

```
=====
EXTRACTED PATTERN RESULTS
=====
```

```
[IPv4 Addresses]
- 192.168.10.15
- 10.10.5.21
```

```
[Email Addresses]
- admin@securitylab.com
```

```
[Session IDs]
- SEC-2026-1045
```

```
[Hexadecimal Sequences]
- a94f3c2d8e71b05f
```